PORTFOLIO PROMPT

Workshop 2 — L'art du prompt

Nom et prénom : Sirine Zouaoui
Groupe / Binôme : groupe 2
Date : 31/10/2026
Thème choisi : langage de programmation python

---

1. Ma tâche

Quelle tâche ai-je demandé à l'IA de réaliser ?

Décrire en quelques lignes la tâche choisie.

Exemple :

Je souhaite utiliser une IA générative pour préparer une explication simple du langage python Destinée à des étudiants de 1ere année LT.

---

2. Première tentative — Prompt V1

Mon premier prompt

Explique Python

Réponse obtenue

Copier ici la réponse de l'IA.

Python est l'un des langages de programmation les plus populaires, puissants et faciles à apprendre au monde. Il est utilisé aussi bien par les débutants que par les plus grandes entreprises (Google, Netflix, NASA, OpenAI).

Voici un résumé clair de tout ce qu'il faut savoir sur Python :

1. Pourquoi Python est-il si populaire ?

- Syntaxe simple et lisible : Son code ressemble à de l'anglais simple. Il y a très peu de ponctuation complexe (pas de point-virgule ; à la fin des lignes, pas d'accolades {}).
- Polyvalent : Vous pouvez presque tout faire avec Python (sites web, intelligence artificielle, analyse de données, jeux, automatisation).
- Grand écosystème de bibliothèques : Il existe des milliers de modules prêts à l'emploi (comme pandas, numpy, matplotlib, PyQt, requests) pour ne pas avoir à tout réinventer.
- Interprété : Le code est exécuté ligne par ligne, ce qui facilite le débogage et le test rapide.

2. À quoi sert Python ?

1. **Intelligence Artificielle & Machine Learning :** C'est le langage numéro 1 pour entraîner des modèles de langue (LLM), la vision par ordinateur et les réseaux de neurones (PyTorch, TensorFlow).
2. **Data Science & Calcul Scientifique :** Analyse de données, visualisation de graphiques et calculs mathématiques complexes (pandas, matplotlib, scipy).
3. Développement Web : Création de serveurs et d'API web (Django, Flask, FastAPI).
4. Logiciels de bureau (GUI) : Création d'interfaces graphiques (PyQt, PySide, Tkinter).
5. **Scripting & Automatisation :** Écrire de petits programmes pour automatiser des tâches répétitives (renommer des fichiers, scraper le web, etc.).

3. Les bases de la syntaxe en Python

A. Les Variables et Types de données

En Python, pas besoin de déclarer le type de variable, il le détecte automatiquement :

Python

```
nom = "Ahmed"        # Chaîne de caractères (str)
age = 20             # Entier (int)
note = 15.5          # Nombre à virgule (float)
est_etudiant = True  # Booleen (bool)
```

B. Les Structures de Contrôle (Conditions)

Python utilise **l'indentation** (les espaces en début de ligne) pour définir les blocs de code :

Python

```
if note >= 10:
    print("Admis !")
else:
    print("Ajourné")
```

C. Les Listes (Arrays)

Pour stocker plusieurs éléments dans une seule variable :

Python

```
langages = ["Python", "PHP", "JavaScript"]
print(langages[0])  # Affiche: Python
```

D. Les Boucles

Pour répéter une action :

PYTHON

```
# Boucle for
for lang in langages:
    print(f"J'apprends {lang}")


# Boucle while
compteur = 0
while compteur < 3:
    print(compteur)
    compteur += 1
```

E. Les Fonctions

Pour regrouper du code réutilisable :

Python

```
def  saluer(nom):
    return f"Bonjour {nom} !"


message = saluer("Ahmed")
print(message)  # Affiche: Bonjour Ahmed !
```

4. Exemple concret : Lire et traiter des données

Python

```
# Exemple de calcul de moyenne
notes = [14, 16, 12, 18, 10]
moyenne = sum(notes) / len(notes)

print(f"La moyenne est de : {moyenne}/20")
```

Si tu souhaites approfondir un sujet précis en Python (comme les interfaces graphiques PyQt/PySide, l'intégration de bases de données, les algorithmes mathématiques ou le Machine Learning), dis-moi ce qui t'intéresse !

Analyse de la réponse

Qu'est-ce qui est satisfaisant ?

Une explication détaillée

Qu'est-ce qui pourrait être amélioré ?

La façon d'expliquer

Qu'est-ce qui manque dans mon prompt ?

Cocher les éléments absents ou insuffisamment précisés :

☐ Rôle*
☐ Contexte*
☐ Tâche précise*
☐ Contraintes*
☐ Format de sortie*
☐ Public cible*
☐ Autre : .....

---

3. Deuxième tentative — Prompt V2

Mon prompt amélioré

Tu joues le rôle d'un professeur qui Explique le language python à un enfant Donne une définition

Précise qui ne dépasse 10 lignes.

Réponse obtenue

Copier ici la réponse de l'IA.

Bonjour ! Sais-tu ce qu'est Python ? C'est un peu comme un super-pouvoir magique, mais pour les ordinateurs.

C'est un **langage de programmation**, c'est-à-dire une langue secrète que l'on utilise pour donner des ordres précis à la machine.

Avec Python, tu peux lui demander de créer tes propres jeux vidéo, de dessiner des formes rigolotes ou même de faire des calculs à ta place.

Il est très facile à apprendre parce que ses mots ressemblent à de l'anglais simple que tout le monde peut comprendre. C'est l'outil parfait pour donner vie à toutes tes idées sur l'écran !

---

4. Comparaison V1 / V2

Critère	V1	V2
Précision	1.5/2	2/2
Pertinence	1/2	2/2
Respect des contraintes	0.5/2	2/2
Respect du format	1/2	2/2
Clarté	1/2	2/2
Total	5/10	10/10

Pourquoi V2 est-elle meilleure ?

Écrire 3 à 5 lignes.

V2 est meilleure car la réponse est plus courte et plus précise dans ce cas on obtient les informations

Nécessaires qu'on a besoin et ne pas perdre du temps à lire. L'utilisation d'une analogie et de contraintes sur le format (exemple commenté) rend la réponse beaucoup plus pédagogique et immédiatement exploitable.

Ai-je réellement obtenu une meilleure réponse ?

☐ Oui*
☐ Non
☐ Partiellement

Justification :

La structure de V2 a forcé l'IA à adapter son niveau de langage et à structurer sa sortie avec des exemples concrets, évitant le jargon technique brut de V1

---

5. Expérience Zero-shot / One-shot / Few-shot

5.1 Zero-shot

Prompt

Écris une fonction Python qui calcule la factorielle d'un nombre

Réponse de l'IA

1. Méthode itérative (Classique et efficace)

C'est la méthode la plus sûre pour éviter les problèmes de dépassement de pile (stack overflow) avec les grands nombres

```
def factorielle_iterative(n):
    if n < 0:
        raise ValueError("La factorielle n'est pas définie pour les nombres négatifs.")
    resultat = 1
    for i in range(2, n + 1):
        resultat *= i
    return resultat


# Exemple d'utilisation :
print(factorielle_iterative(5))  # Affiche 120
```

2. Méthode récursive (Élégante)

Cette approche utilise la définition mathématique de la factorielle ($n! = n \times (n-1)!$).

```
def factorielle_recursive(n):
    if n < 0:
        raise ValueError("La factorielle n'est pas définie pour les nombres négatifs.")
    if n == 0 or n == 1:
        return 1
    return n * factorielle_recursive(n - 1)


# Exemple d'utilisation :
print(factorielle_recursive(5))  # Affiche 120
```

3. Utilisation du module intégré `math` (Recommandé pour la production)

Si vous travaillez sur un projet réel, Python propose une fonction native ultra-rapide et optimisée dans le module

```
import math


n = 5
resultat = math.factorial(n)
```

print(resultat) # Affiche 120

5.2 One-shot

Prompt

Écris une fonction Python avec des commentaires pédagogiques. Exemple : # Fonction pour additionner deux nombres def addition(a, b): return a + b Maintenant, écris une fonction pour calculer la factorielle d'un nombre.

Réponse de l'IA

Fonction pour calculer la factorielle d'un nombre (produit de tous les entiers de 1 à n)

def factorielle(n): return 1 if n <= 1 else n * factorielle(n - 1)

5.3 Few-shot

Prompt

Fournis des fonctions Python avec leur type de retour et un exemple d'appel documenté. Exemple 1 : def carre(x: int) -> int: return x * x # Test: carre(4) -> 16 Exemple 2 : def est_pair(n: int) -> bool: return n % 2 == 0 # Test: est_pair(3) -> False Maintenant, écris une fonction pour calculer la factorielle d'un nombre selon ce même format.

Réponse de l'IA

def factorielle(n: int) -> int: return 1 if n <= 1 else n * factorielle(n - 1) # Test: factorielle(5) -> 120.

Comparaison

Qu'avez-vous remarqué entre les trois approches ?

Le *zero-shot* donne une réponse standard. Le *one-shot* et le *few-shot* permettent de cadrer précisément le style, le format et la structure attendus grâce à des exemples concrets.

Selon vous, dans **quelle situation le few**-shot est-**il utile ?**

Il est très utile pour formater des données (ex: tableaux), respecter des conventions de code strictes ou imiter un ton rédactionnel particulier.

---

6. Auto-**critique légère**

J'ai demandé à l'IA :

Analyse ta réponse précédente. Identifie deux points forts, deux faiblesses et propose deux améliorations.

Résultat de l'auto-**critique**

Points forts : Clarté de l'analogie, exactitude du code. Faiblesses : Manque de cas limites (ex: nombres négatifs), pas de gestion des erreurs. Améliorations : Ajouter une vérification des entrées et un test avec une boucle `while`

Est-ce que ce**tte auto-critique garantit que la réponse est correcte ?**

☐ Oui
☐ Non*

Pourquoi ?

L'IA peut passer à côté d'erreurs logiques subtiles ou de failles de sécurité dans son propre code. L'auto-critique est générée par le même modèle, elle ne remplace pas une exécution réelle ou une relecture humaine

Rappel :

L'auto-critique de l'IA est une aide à l'analyse. Elle ne remplace pas la vérification humaine.

---

7. Mon prompt **fi**nal

Après les différentes expériences, rédiger une version finale améliorée.

Tu agis en tant qu'expert en didactique de la programmation Python. Le contexte est l'apprentissage des bases pour des étudiants de première année. Ta tâche est d'expliquer l'utilisation et la syntaxe d'une boucle `for` en Python. Respecte les contraintes suivantes : utilise une analogie simple, fournis un exemple commenté pas à pas, et inclus un cas d'usage concret en télécommunications ou traitement de données basique. Présente la sortie sous forme de sections claires (Définition, Analogie, Code, Exemple d'application).

Quels éléments ai-je ajoutés ou modi**fi**és ?

☐ Rôle*
☐ Contexte*
☐ Tâche*
☐ Contraintes*
☐ Format*
☐ Exemples / few-shot*
☐ Critères de qualité*

☐ Vérification*
☐ Autre : .....

---

8. Mon gabarit de prompt réutilisable

Construire ici un modèle que vous pourrez réutiliser dans d'autres situations.

RÔLE

Tu es un expert et pédagogue dans ton domaine.

CONTEXTE

Le contexte est la préparation d'un support pédagogique ou technique pour des débutants.

TÂCHE

Ta tâche est d'expliquer clairement un concept technique et de fournir un cas pratique.

CONTRAINTES

Respecte les contraintes suivantes :

- Utiliser un langage clair, sans jargon non expliqué
- Ne pas omettre les cas particuliers ou limites
- Rester concis et structuré

FORMAT

Présente la réponse sous la forme suivante :
Présente la réponse sous la forme suivante : Introduction, Analogie/Explication, Exemple de code/cas pratique, Résumé.

CRITÈRES DE QUALITÉ

La réponse doit être :

- Pédagogique
- Rigoureuse techniquement
- Facilement compréhensible en moins de 3 minutes

VÉRIFICATION

Indique les éléments incertains ou nécessitant une vérification. N'invente pas de source.

---

9. Ma réflexion personnelle

Répondre brièvement aux questions suivantes.

1. Quel élément du prompt a eu le plus d'impact sur la réponse ?

L'ajout du rôle et du public cible, car cela oblige l'IA à calibrer son niveau de complexité et de vocabulaire

2. Quelle erreur ai-je commise dans mon premier prompt ?

Il était trop minimaliste et directif, sans préciser le format attendu ni le niveau de l'interlocuteur

3. Qu'ai-je appris en comparant V1 et V2 ?

Qu'un prompt structuré (méthode des blocs) réduit drastiquement les allers-retours et donne directement un résultat de qualité professionnelle

4. Quand le few-shot est-**il utile ?**

Il est indispensable lorsqu'on souhaite imiter un format de sortie très spécifique ou un style d'écriture particulier.

5. Pourquoi faut-il vérifier une réponse produite par une IA ?

Parce que l'IA peut produire des hallucinations ou des erreurs subtiles (dans le code ou les faits) tout en formulant sa réponse avec une grande assurance.

6. Dans quel autre domaine pourrais-**je réutiliser mon gabarit ?**

Pour rédiger des résumés de cours, structurer des documentations de projets de programmation (Python/C) ou préparer des synthèses de recherches documentaires.

---

10. Auto-**évaluation avant** remise

Critère	Oui	**Partiellement**	Non
Mon rôle est clairement défini	*☐	☐	☐
Mon contexte est suffisamment précis	*☐	☐	☐
Ma tâche est clairement formulée	*☐	☐	☐
Mes contraintes sont explicites	*☐	☐	☐
Le format de sortie est précisé	*☐	☐	☐
J'ai testé au moins deux versions du prompt	*☐	☐	☐
J'ai comparé les résultats	*☐	☐	☐
J'ai expérimenté zéro-shot / one-shot / few-shot	*☐	☐	☐
J'ai utilisé l'auto-critique de manière critique	*☐	☐	☐

Critère	Oui	**Partiellement**	Non
J’ai construit un gabarit réutilisable	*☐	☐	☐
Je peux expliquer les améliorations apportées	*☐	☐	☐

À retenir

Un bon prompt ne consiste pas seulement à poser une question.

Rôle + Contexte + Tâche + Contraintes + Format

Et surtout :

Je teste → j’évalue → j’améliore → je vérifie.